Xi'an Jiaotong-Liverpool University
西交利物浦大學

# Introduction to Database

Welcome to CPT103!

# Table of Contents


- Module information
- Introduction to database
- Relational Model
- Relational Keys

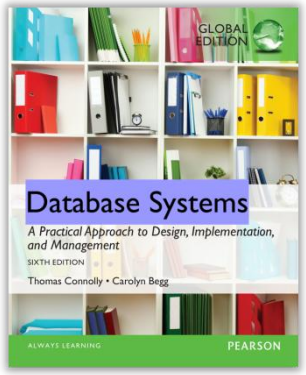
# Module Information

About the module instructor, teaching organisation

# Teaching Organisation

- Module instructors:
  - Jianjun Chen (Jianjun.Chen@xjtlu.edu.cn) SD541
  - Jun Qi (Jun.Qi@xjtlu.edu.cn) SD461
  - Yu Liu (Yu.Liu02@xjtlu.edu.cn) SD465

  - Office hours: please check the module page online




- Textbook: "Database Systems: a practical approach to design, implementation, and management"
  by Connolly, Thomas M., Begg, Carolyn E.

# Teaching Organisation

- Support:
  - Discussions with classmates enable you to better understand concepts and terminologies.
  - I will also send notifications to you about any updates. Please check emails frequently.
  - Lab sessions are supported by TA and module instructors.

- Assessments: 15% online quiz and 85 % closed-book exam.
  - Quiz time will be announced in lectures and will be sent to you via notification.
  - Exam time can be found from the university timetable (end of semester).

# What Do You Need?

- Lectures:
  - A piece of note that covers all important knowledge.
  - Laptop or tablets: Some of the questions are on the LearningMall.
  - A friend that you can discuss with during breaks.

- Labs:
  - Someone to discuss with.
  - Questions you want to ask to me.

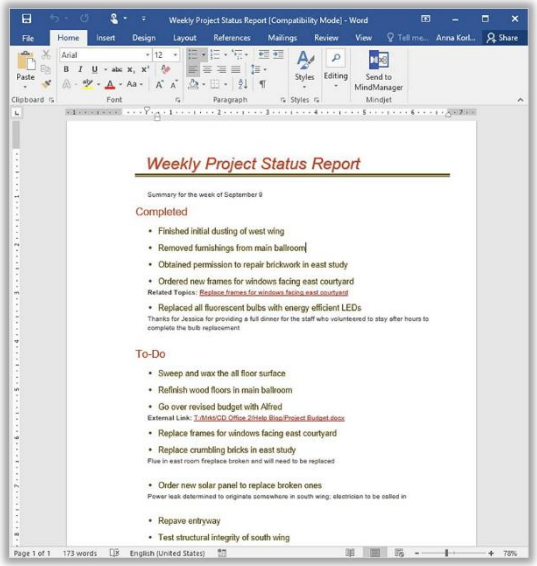
- Assessments:
  - Revise contents weekly.
  - Do not check slides only at the last moment!

# Introduction to Database Systems

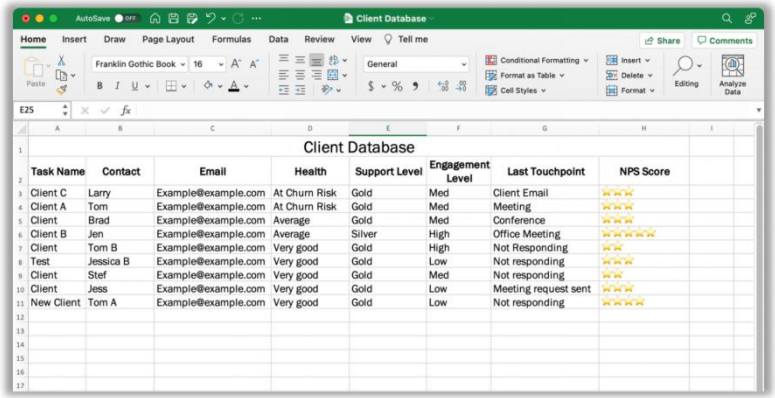
What is a database? What is a database management system?

# What is Data?

- **Example 1**: Data could be a docx file storing your project status report.
  - Can be opened using Word.




- **Example 2**: Data could be a spreadsheet containing patients' information.
  - Can be opened using Excel



# What is Data?

- Data is only meaningful under its designed scenario.
  - **In example 1:** Tracking and reporting project status.
  - **In example 2:** Storing patient information for retreival later.
- Data must have the means to be created/modified.
- Data must have the means to be accessed.

# What is Database?

- **Database**: “Organised collection of data. Structured, arranged for ease and speed of search and retrieval.”

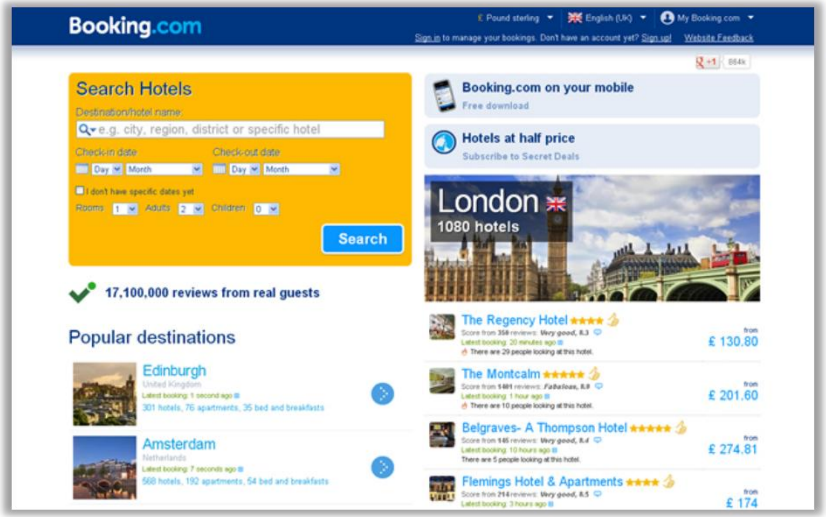
- **Database Management System (DBMS)**: Software that is designed to enable users and programs to store, retrieve and update data from database.

# Database Applications

- Mobile Payment: account balance, transaction information are stored in databases.

Pay 支付宝 ALIPAY

- Hotel booking website: guest information, booking history, hotel information etc..



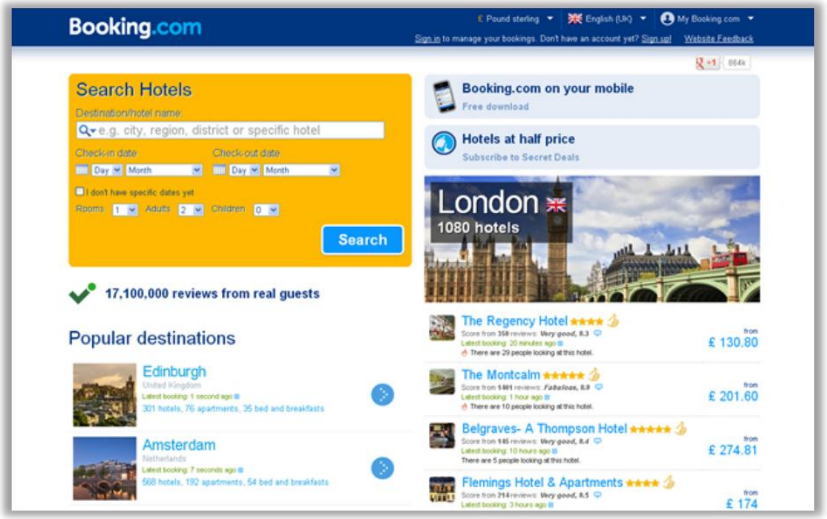
# Database Applications

- Online game data storage: account names, skins and equipment information of gamers are stored in databases.

- Healthcare: patient information is stored in databases.

# Question

- Can you identify other information that could be stored in database?



# Components of the DBMS Environment

- **Hardware**: the computer where the DBMS runs on.
  - Can be a personal computer, a mainframe, or a network of computers.
- **Software**: comprises the DBMS itself, application programs (such as Alipay) and the operating system.
- **Data**: comprises both operational data and metadata.
  - Operational data: e.g., game account information of a game company.
  - Metadata: also know as "data about data".
- **Procedures**: the instructions and rules that govern the design and use of the database.
  - Such as "how to backup database"
- **People**: database designers, end users, application developers and database administrators.

# DBMS Functions / Must Haves

1. Allow users to store, retrieve and update data.
2. A “system catalog” that holds hold data about the schemas, users, applications, and so on. (Also called metadata)
3. Ensure either that all the updates corresponding to a given action are made or that none of them is made (Atomicity).
4. Ensure that DB is updated correctly when multiple users are updating it concurrently.
5. Recover the DB in the event it is damaged in any way.
6. Ensure that only authorised users can access the DB.
7. DBMS should be accessible from remote computers over a network.
8. Be capable of integrating with other software.

*all underlined points will be covered (more or less) in this module.

# Data Models

"Organised collection of data. Structured, arranged for ease and speed of search and retrieval."

- The structure may follow different models.
  - **Relational Data Model**
  - Hierarchical Data Model
  - Network Data Model

Branch

| branchNo | street | city | postCode |
|---|---|---|---|
| B005 | 22 Deer Rd | London | SW1 4EH |
| B007 | 16 Argyll St | Aberdeen | AB2 3SU |
| B003 | 163 Main St | Glasgow | G11 9QX |
| B004 | 32 Manse Rd | Bristol | BS99 1NZ |
| B002 | 56 Clover Dr | London | NW10 6EU |

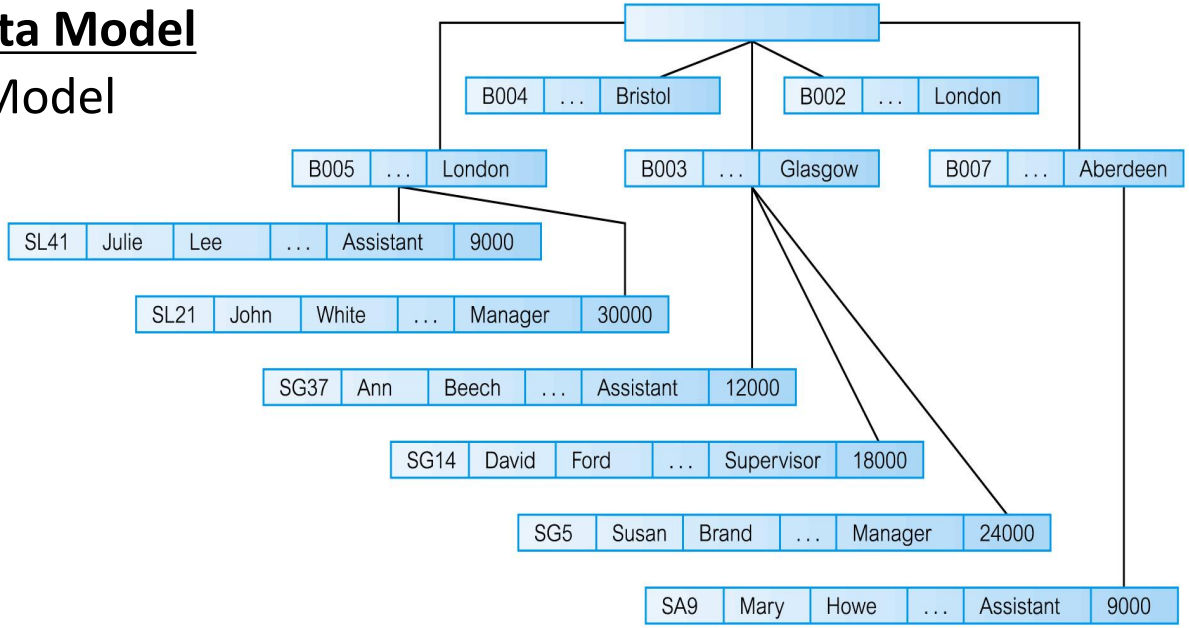
Staff

| staffNo | fName | lName | position | sex | DOB | salary | branchNo |
|---|---|---|---|---|---|---|---|
| SL21 | John | White | Manager | M | 1-Oct-45 | 30000 | B005 |
| SG37 | Ann | Beech | Assistant | F | 10-Nov-60 | 12000 | B003 |
| SG14 | David | Ford | Supervisor | M | 24-Mar-58 | 18000 | B003 |
| SA9 | Mary | Howe | Assistant | F | 19-Feb-70 | 9000 | B007 |
| SG5 | Susan | Brand | Manager | F | 3-Jun-40 | 24000 | B003 |
| SL41 | Julie | Lee | Assistant | F | 13-Jun-65 | 9000 | B005 |

# Data Models

"Organised collection of data. Structured, arranged for ease and speed of search and retrieval."

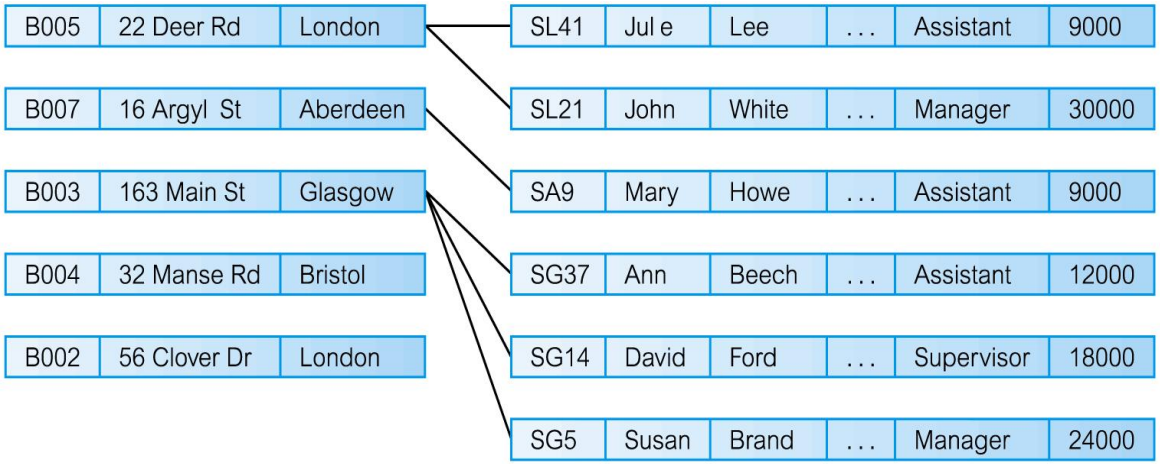
- The structure may follow different models.
  - Relational Data Model
  - **Hierarchical Data Model**
  - Network Data Model



# Data Models

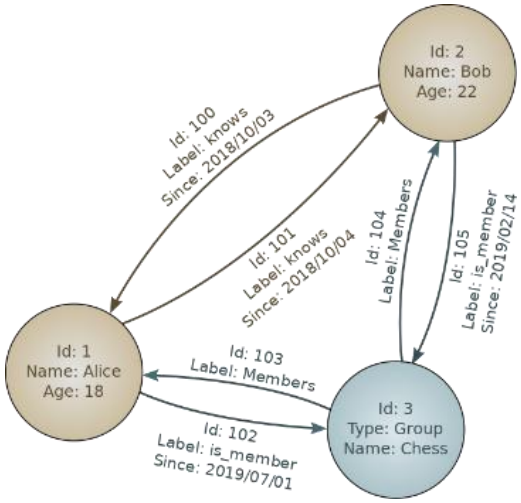
"Organised collection of data. Structured, arranged for ease and speed of search and retrieval."

- The structure may follow different models.
  - Relational Data Model
  - Hierarchical Data Model
  - **Network Data Model**



# Other Data Models

- Other data models:
  - **<u>Entity-Relationship model</u>**: No databases directly uses this model. It is now commonly used for teaching students the basics of database structure.
  - **Graph model**: uses graph structures for semantic queries with nodes, edges
  - **Document model**: stores documents in the form of JSON, XML etc..




```
{
    "FirstName": "Bob",
    "Address": "5 Oak St.",
    "Hobby": "sailing"
}
```

  - Documents are fetched using keys.
  - **Object-Oriented**: information is presented as objects as used in object-oriented programming.

# Optional Reading

- For additional contents, you may read Chapters 1 ~ 3.
  - Some contents can be hard to fully understand now.

# Relational Model

Terminologies, properties

# The Relational Model

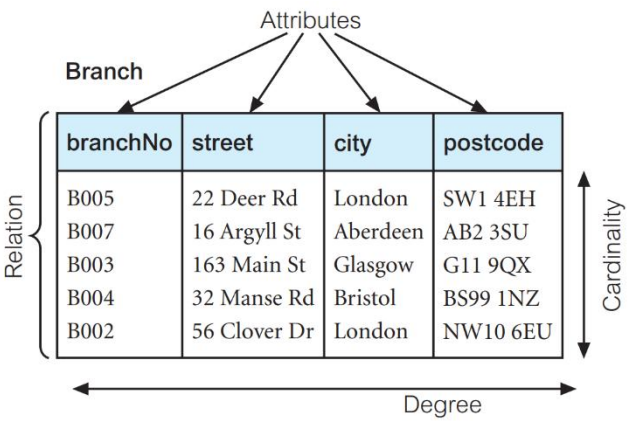
- The relational model is **one approach** to managing data.
  - Originally Introduced by E.F. Codd in his paper "A Relational Model of Data for Large Shared Databanks", 1970.

- The model uses a structure and language that is consistent with first-order predicate logic
  - Provides a declarative method for specifying data and queries
  - Details are covered in the Chapter 4~5 of the textbook.

# Terminologies

- A relation is a mathematical concept. The physical form of a relation is a table with columns and rows.

- An attribute is a named column of a relation.
  属性是关系的命名列（每一列的名称）
- A domain is the set of allowable values for attributes.
  域是所有可能取值的集合



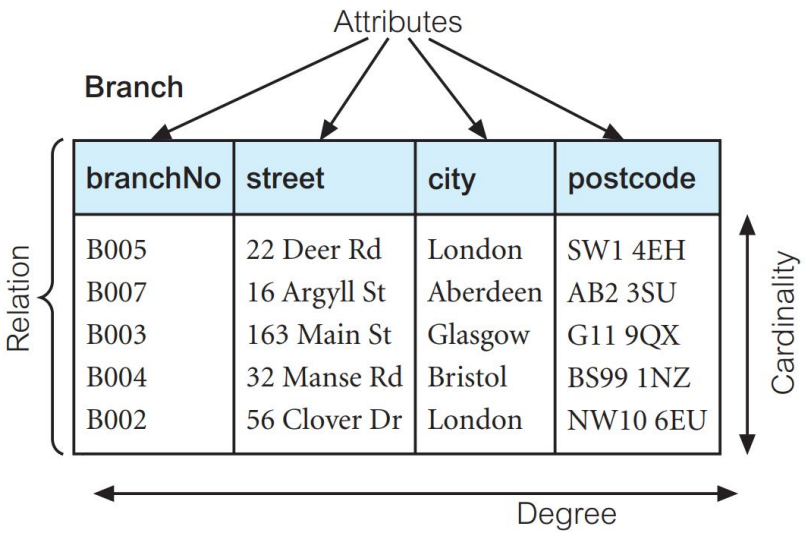

| branchNo | street | city | postcode |
|---|---|---|---|
| B005 | 22 Deer Rd | London | SW1 4EH |
| B007 | 16 Argyll St | Aberdeen | AB2 3SU |
| B003 | 163 Main St | Glasgow | G11 9QX |
| B004 | 32 Manse Rd | Bristol | BS99 1NZ |
| B002 | 56 Clover Dr | London | NW10 6EU |

| Attribute | Domain Name | Meaning | Domain Definition |
|---|---|---|---|
| branchNo | BranchNumbers | The set of all possible branch numbers | character: size 4, range B001–B999 |
| street | StreetNames | The set of all street names in Britain | character: size 25 |
| city | CityNames | The set of all city names in Britain | character: size 15 |
| postcode | Postcodes | The set of all postcodes in Britain | character: size 8 |
| sex | Sex | The sex of a person | character: size 1, value M or F |
| DOB | DatesOfBirth | Possible values of staff birth dates | date, range from 1-Jan-20, format dd-mmm-yy |
| salary | Salaries | Possible values of staff salaries | monetary: 7 digits, range 6000.00–40000.00 |

# Terminologies

元组是关系（表）中的一行数据，每个元组代表一个完整的记录

- Tuple: a tuple is a row of a relation.
  - Mathematically, the order of tuples does not matter.
- The degree of a relation is the number of attributes it contains.
- Cardinality: the number of tuples in a relation.




| branchNo | street | city | postcode |
|---|---|---|---|
| B005 | 22 Deer Rd | London | SW1 4EH |
| B007 | 16 Argyll St | Aberdeen | AB2 3SU |
| B003 | 163 Main St | Glasgow | G11 9QX |
| B004 | 32 Manse Rd | Bristol | BS99 1NZ |
| B002 | 56 Clover Dr | London | NW10 6EU |

度是关系中包含的属性（列）的数量
基数是关系中元组的数量

# Alternative Terminologies

| Formal Terms | Alternative #1 | Alternative #2 |
|---|---|---|
| Relation | Table | File |
| Tuple | Row | Record |
| Attribute | Column | Field |

# Question

- Can you point out what the previous terminologies refer to in this table?

Staff

| ID | Name | Salary | Department |
|---|---|---|---|
| M139 | John Smith | 18000 | Marketing |
| M140 | Mary Jones | 22000 | Marketing |
| A368 | Jane Brown | 22000 | Accounts |
| P222 | Mark Brown | 24000 | Personnel |
| A367 | David Jones | 20000 | Accounts |

# Question

Attributes : ID, Name, Salary & Department

The degree of the relation is 4

Staff

| ID | Name | Salary | Department |
|---|---|---|---|
| M139 | John Smith | 18000 | Marketing |
| M140 | Mary Jones | 22000 | Marketing |
| A368 | Jane Brown | 22000 | Accounts |
| P222 | Mark Brown | 24000 | Personnel |
| A367 | David Jones | 20000 | Accounts |

Tuples

The cardinality of the relation is 5

# Additional Properties of Relations

- Relation's name is unique in the relational database schema.
- Each cell contains exactly one atomic value.
- Each attribute of a relation must have a distinct name.
- The values of an attribute are from the same domain.
- The order of attributes has no significance.
- The order of tuples has no significance.
- No duplicate tuples.

# Relation and Tuples

- One relation stands for one class/type of objects.
- Each tuple is an instance of that class/type.
- In the example below, each tuple of staff refers to a real staff.
  - Each tuple has a value in ID, Name, Salary and Department.

Staff

| ID | Name | Salary | Department |
|---|---|---|---|
| M139 | John Smith | 18000 | Marketing |
| M140 | Mary Jones | 22000 | Marketing |
| A368 | Jane Brown | 22000 | Accounts |
| P222 | Mark Brown | 24000 | Personnel |
| A367 | David Jones | 20000 | Accounts |

# Relational Keys

Super key, candidate key, primary key, foreign key.

# Relation Keys

- No duplicate tuples are allowed within a relation.
- Thus, we need to be able to identify one or more attributes (called relational keys) that uniquely identifies each tuple in a relation.
- *Examples:*

Staff

| ID | Name | Age |
|---|---|---|
| 1 | Jason | 22 |

Office

| Building Number | Room | Room Size | Has Printer |
|---|---|---|---|
| 11 | 301 | 96 | True |

# “Uniquely Identifies”

- Given two staff members in a company.
  - Their names could be the same.
  - Their ages could be the same.
  - But their ID numbers are always different. The ID is unique to the person.

Staff

| ID | Name | Age |
|---|---|---|
| 1 | Jason | 22 |
| 2 | John | 31 |
| 3 | Joson | 31 |

# “Uniquely Identifies”

- Given two office rooms in a company.
  - Their building number could be the same.
  - Their room numbers could be the same.
  - Rooms with the same room size are extremely common.
  - But the combination of “Building number + Room” is unique.

Office

| Building Number | Room | Room Size | Has Printer |
|---|---|---|---|
| 11 | 301 | 96 | True |
| 11 | 502 | 96 | False |
| 12 | 301 | 32 | False |
| 12 | 502 | 32 | True |

# Questions

- Which of the following attribute combinations uniquely identifies offices?
  1. (Building Number, Room Size)
  2. (Room, Room Size)
  3. (Building Number, Room, Has Printer)
  4. (Building Number, Room, Room Size)
  5. (Building Number, Room, Room Size, Has Printer)

Office

| Building Number | Room | Room Size | Has Printer |
|---|---|---|---|
| 11 | 301 | 96 | True |
| 11 | 502 | 96 | False |
| 12 | 301 | 32 | False |
| 12 | 502 | 32 | True |

# Relation Keys

- **Superkey**: An attribute, or set of attributes, that uniquely identifies a tuple within a relation.
  - *A superkey may contain additional attributes that are not necessary for unique identification.*
- **Candidate key**: A superkey such that no proper subset is a superkey within the relation.
  - *There may be several candidate keys for a relation.*
- **Primary key**: The candidate key that is selected to identify tuples uniquely within the relation.
  - The candidate keys that are not selected to be the primary key are called **alternate keys**.
  - A relation can only have one primary key.
- When a key consists of more than one attribute, we call it a **composite key.**

# 键的层次结构

## 超键（Superkey）

- 超键是能够唯一标识关系中每个元组的属性或属性组合
- 只要能区分表中的每一行记录，就是超键
- 例如：在学生表中，{学号}、{学号,姓名}、{学号,姓名,年龄} 都可能是超键
- 超键可能包含多余的属性

## 候选键（Candidate Key）

- 候选键是最小的超键，即去掉任何一个属性后就不再是超键
- 它不包含多余的属性，是"精简版"的超键
- 一个关系可能有多个候选键
- 例如：如果{学号}和{身份证号}都能唯一标识学生，那么它们都是候选键

## 主键（Primary Key）

- 主键是从候选键中选择出来的，用于唯一标识元组的键
- 每个关系只能有一个主键
- 主键的值不能为空（非空约束）
- 主键一旦确定，通常不会改变

## 替代键（Alternate Keys）

- 替代键是那些没有被选为主键的候选键
- 它们同样具有唯一标识能力，但在设计中作为备选方案

## 复合键（Composite Key）

- 当键由多个属性组成时，称为复合键
- 例如：{课程号, 学号} 组合起来唯一标识选课记录

## 举例说明

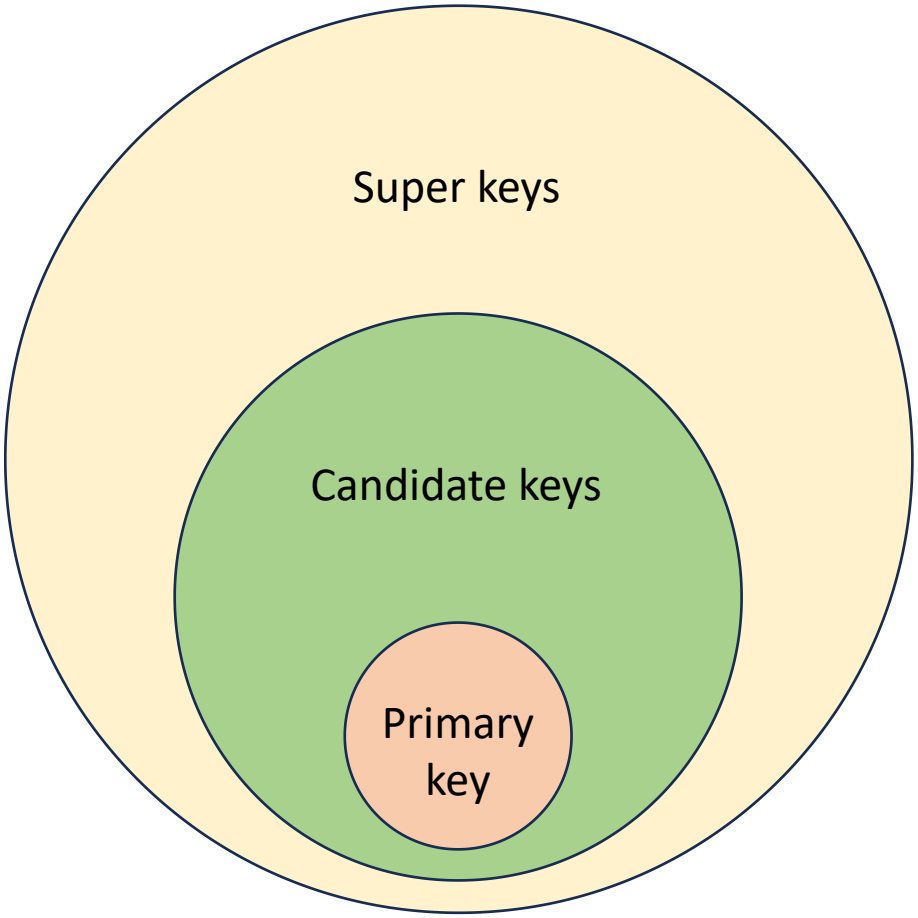
假设学生表有：学号、身份证号、姓名、年龄

- **超键**：{学号}、{身份证号}、{学号,姓名}、{身份证号,年龄} 等
- **候选键**：{学号}、{身份证号}（都是最小超键）
- **主键**：选择 {学号} 作为主键
- **替代键**：{身份证号}
- **复合键**：如果需要用{姓名,年龄}组合来标识，这就是复合键

这种层次化的键概念确保了数据的唯一性和完整性。

必须能唯一标识元组且去掉任何一个属性都不能唯一标识（不能有多余的）

# Relation Keys



# Question

- Identify the type of relation key of the following items for table 'teaching_staff':
  1. (staff_id, email)
  2. (staff_id)
  3. (passport_id)
  4. (passport_id, fname)
  5. (staff_id, passport_id)
  6. (email, lname)
  7. (fname, lname)

superkey

candidate key

primary key

alternate key

composite key

teaching_staff

| staff_id | email | fname | lname | passport_id |
|---|---|---|---|---|
| 1 | S.Guan@xjtlu.edu.cn | Steven | Guan | P123456 |
| 2 | J.Woodward@nott.ac.uk | John | Woodward | U543121 |
| 3 | N.Tubb@bhan.ac.uk | Nathan | Tubb | U998877 |

# Answer

1. (staff_id, email) superkey, composite key
2. (staff_id) superkey, candidate key, primary key
3. (passport_id) superkey, candidate key, alternate key
4. (passport_id, fname) superkey, composite key
5. (staff_id, passport_id) superkey, composite key
6. (email, lname) superkey, composite key
7. (fname, lname) ---none---

teaching_staff

| staff_id | email | fname | lname | passport_id |
|---|---|---|---|---|
| 1 | S.Guan@xjtlu.edu.cn | Steven | Guan | P123456 |
| 2 | J.Woodward@nott.ac.uk | John | Woodward | U543121 |
| 3 | N.Tubb@bhan.ac.uk | Nathan | Tubb | U998877 |

# Finding Candidate Key

- You can't necessarily infer the candidate keys based solely on the data in your table.
  - More often than not, an instance of a relation will only hold a small subset of all the possible values.
  - E.g. Restaurants' booking number might reset to 1 after a large number.

Queue No. A31
Table size: up to 4

31 People are waiting ahead of you.

A1, A2, A3… A99, A999 -> A1

Queue

| queue_no | table_size | phone_no | ticket_time |
|---|---|---|---|
| A1 | 4 | 123321 | 2021-9-10 12:00:03 |
| A2 | 4 | 321123 | 2021-9-10 12:15:06 |
| B1 | 8 | 789987 | 2021-9-10 12:16:09 |

# Finding Candidate Key

- What is a suitable candidate key for the Queue table then?

Queue No. A31
Table size: up to 4

31 People are waiting ahead of you.

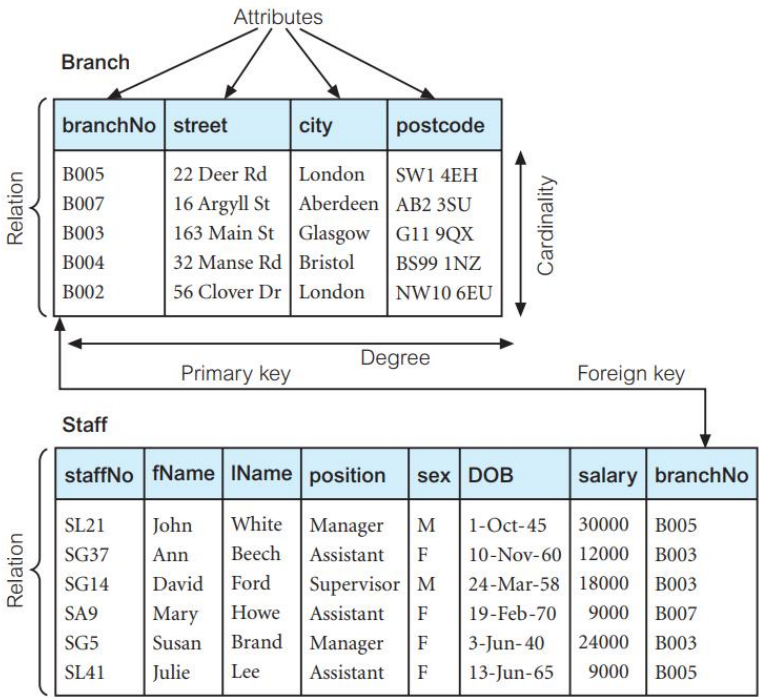
A1, A2, A3… A99, A999 -> A1

Queue

| queue_no | table_size | phone_no | ticket_time |
|---|---|---|---|
| A1 | 4 | 123321 | 2021-9-10 12:00:03 |
| A2 | 4 | 321123 | 2021-9-10 12:15:06 |
| B1 | 8 | 789987 | 2021-9-10 12:16:09 |

# Relation Keys: Foreign Key

- It is also very common that tuples in one relation references data from another relation.
  - As a result, a database should provide such mechanism to ensure correct references.
- This is enforced by something called **foreign key**



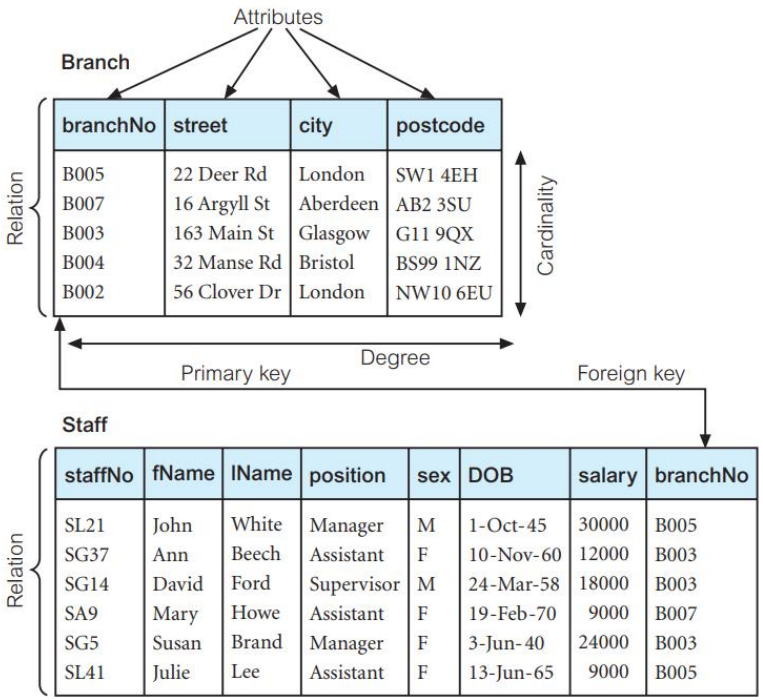

| branchNo | street | city | postcode |
|---|---|---|---|
| B005 | 22 Deer Rd | London | SW1 4EH |
| B007 | 16 Argyll St | Aberdeen | AB2 3SU |
| B003 | 163 Main St | Glasgow | G11 9QX |
| B004 | 32 Manse Rd | Bristol | BS99 1NZ |
| B002 | 56 Clover Dr | London | NW10 6EU |

| staffNo | fName | lName | position | sex | DOB | salary | branchNo |
|---|---|---|---|---|---|---|---|
| SL21 | John | White | Manager | M | 1-Oct-45 | 30000 | B005 |
| SG37 | Ann | Beech | Assistant | F | 10-Nov-60 | 12000 | B003 |
| SG14 | David | Ford | Supervisor | M | 24-Mar-58 | 18000 | B003 |
| SA9 | Mary | Howe | Assistant | F | 19-Feb-70 | 9000 | B007 |
| SG5 | Susan | Brand | Manager | F | 3-Jun-40 | 24000 | B003 |
| SL41 | Julie | Lee | Assistant | F | 13-Jun-65 | 9000 | B005 |

# Foreign Key

- **Foreign key**:
  - One or more attributes within one relation that **must match** the candidate key of some (possibly the same) relation.

- Example:
  - We want the values of the 'branchNo' in relation <u>staff</u> to be one of the 'branchNo' in relation <u>Branch</u>.




| branchNo | street | city | postcode |
|---|---|---|---|
| B005 | 22 Deer Rd | London | SW1 4EH |
| B007 | 16 Argyll St | Aberdeen | AB2 3SU |
| B003 | 163 Main St | Glasgow | G11 9QX |
| B004 | 32 Manse Rd | Bristol | BS99 1NZ |
| B002 | 56 Clover Dr | London | NW10 6EU |

| staffNo | fName | lName | position | sex | DOB | salary | branchNo |
|---|---|---|---|---|---|---|---|
| SL21 | John | White | Manager | M | 1-Oct-45 | 30000 | B005 |
| SG37 | Ann | Beech | Assistant | F | 10-Nov-60 | 12000 | B003 |
| SG14 | David | Ford | Supervisor | M | 24-Mar-58 | 18000 | B003 |
| SA9 | Mary | Howe | Assistant | F | 19-Feb-70 | 9000 | B007 |
| SG5 | Susan | Brand | Manager | F | 3-Jun-40 | 24000 | B003 |
| SL41 | Julie | Lee | Assistant | F | 13-Jun-65 | 9000 | B005 |